

Verteilte Systeme

Aufgabe 2

1

1

Einleitung

In dieser Aufgabe ist bitte ein einfacher verteilter Algorithmus und dessen Verwaltung/Koordination zu implementieren. Jeder Arbeitsprozess (ggT-Prozess) durchläuft denselben Algorithmus! Mit diesem Algorithmus kann man z.B. mit 42 Arbeitsprozessen den ggT von 42 Zahlen nebenläufig bestimmen. Es sei aber darauf hingewiesen, dass es in dieser Aufgabe nicht darum geht, besonders effizient den ggT von vielen Zahlen zu berechnen! Das Szenario dient vorwiegend dem Zweck, Phänomene in verteilten Systemen im Falle eines verteilten Algorithmus zu veranschaulichen.

Beachten Sie: Alle benötigten Hilfsfunktionen sind zu Lernzwecken selbst zu implementieren (ohne Verwendung von KI-Tools). Insbesondere sind List-Comprehensions (`[...|...]`), Indexzugriffe auf Tupel oder Listen sowie die Verwendung des Moduls `lists` (oder anderer Standardmodule) nicht zulässig. Auch die Verwendung höherer Ordnungsfunktionen oder deren Eigenimplementierung – etwa `foreach/2`, `keyfind/3`, `flatmap/2` oder `mapfoldl/3` – ist nicht erlaubt. Der Zugriff auf Listen und Tupel ist ausschließlich per Pattern Matching (z. B. `[Head | Tail]`) gestattet. Algorithmen auf den Basisstrukturen müssen Sie daher vollständig selbst implementieren. Die Verwendung anderer, komplexerer Datenstrukturen wie z. B. `dict` oder `sets` ist ebenfalls nicht zulässig. Fragen Sie bitte im Zweifel unbedingt nach!

Bei Einsatz nicht erlaubter Funktionen wird die Abgabe als fehlerhaft gewertet.

2

2

Aufgabenstellung

Das System des verteilten Algorithmus ist so ausgelegt, dass es für eine unbestimmte Betriebszeit gestartet wird, um mehrere ggT-Berechnungen durchführen zu können. Die zu implementierende Terminierungsabstimmung bezieht sich daher nur auf eine Berechnung, nicht auf die Betriebszeit des Systems.

Für die Implementierung werden im Wesentlichen drei Module benötigt:

- **Der Koordinator**, der den verteilten Algorithmus verwaltet und eine Art Monitor ist. Dazu gehören das hochfahren des Systems, den Start einer ggT-Berechnung und das herunterfahren des Systems. Der Koordinator verfügt über eine GUI (Textausgabe auf dem Bildschirm und log-Dateien), in der der Ablauf des Algorithmus beobachtet werden kann. Der Koordinator wird wie der unterstützende Namensdienst manuell gestartet.
- **Der Starter**, der im Wesentlichen das Starten der ggT-Prozesse auf für den Koordinator entfernten und damit zunächst nicht zugänglichen Rechnern ermöglicht. Die Starter sind auf den entfernten Rechnern manuell zu starten.
- **Der ggT-Prozess**, der die eigentliche Arbeit leistet, also die verteilte Berechnung des ggT durchführt. ggT-Prozesse werden durch den Starter gestartet.

Alle Module erstellen ausführliche und eindeutig ihnen zugeordnete Logdateien. Insbesondere hat jeder ggT-Prozess eine eigene log-Datei! Zur Kommunikation wird das Message-Passing von Erlang/OTP verwendet, d.h. die Kommunikation ist asynchron.

3

3

Aufgabenstellung

Der Algorithmus: Der Algorithmus basiert auf dem Satz von Euklid: Der größte gemeinsame Teiler (ggT) zweier positiver ganzer Zahlen x, y (mit $x \geq y > 0$) ist gleich dem ggT von y und dem Rest, der bei ganzzahliger Division von x durch y entsteht.

Der verteilte Algorithmus lautet:

- Jeder Prozeß P_i hat seine eigene Variable M_i , in der die von ihm zu verwaltende Zahl steht. Alle Prozesse sind in einem Ring angeordnet.
- Der ggT aller am Anfang bestehender M_i wird wie folgt berechnet:

```

{Eine Nachricht <y> ist eingetroffen}
  if  y < Mi then Mi := mod(Mi-1,y)+1;
                                send #Mi to all neighbours;
  fi

```

Initialisierungsphase: Nachdem der Namensdienst und der Koordinator gestartet wurden, werden die Starter gestartet und erfragen beim Koordinator die steuernden Werte für die ggT-Prozesse (`getsteeringval`). Die ggT-Prozesse werden gemäß diesen Vorgaben des Koordinators von den Startern gestartet. Die ggT-Prozesse registrieren sich lokal bei der Erlang-Node und beim Namensdienst (`rebind|bind`) und sie melden sich beim Koordinator an (`hello`). Der Koordinator baut einen (zufällig gemischten) Ring aus den angemeldeten ggT-Prozessen auf und informiert die ggT-Prozesse über ihre (linken und rechten) Nachbarn (`setneighbors`). Das System ist nun bereit, ggT-Aufgaben für so viele Zahlen zu lösen, wie es ggT-Prozesse gibt.

4

4

Aufgabenstellung

Arbeitsphase: Für eine ggT-Berechnung wird das System initialisiert: Der Koordinator informiert die ggT-Prozesse über ihre Werte $\#M_i$ (`setpm`). Er wählt per Zufall 20% aller ggT-Prozesse (mindestens jedoch zwei) aus, die mit der Berechnung beginnen sollen. Dazu sendet er diesen die Auswahlinformation zu (`calc, start`). Der jeweils ausgewählte ggT-Prozess erfragt dann den initialen Wert (`getinit`). Verändert sich bei einem ggT-Prozess auf Grund der (zeitverzögernden) Berechnung sein Wert $\#M_i$, so informiert er den Koordinator darüber (`briefmi`) und seinen linken und rechten Nachbarn (`sendy`). Stellt ein ggT-Prozess die Terminierung der aktuellen Berechnung durch Abstimmung mit seinen Nachbarn fest, so meldet er dem Koordinator das (evtl. falsche) Endergebnis der Berechnung (`briefterm`). Achtung: nach seiner Terminierungsmeldung oder auch während der Abstimmungsphase kann dieser ggT-Prozess mittels `sendy` zur Fortführung der aktuellen Berechnung gebracht werden! Ist der aktuelle Berechnungsprozess abgeschlossen (ggT wurde ermittelt), steht das System für weitere ggT-Berechnungen zur Verfügung.

Erhält während einer Berechnung ein ggT-Prozess $?x_{term}$ Sekunden lang keine Zahl y (`sendy` oder `setpm`), startet er über den Namensdienst eine Terminierungsabstimmung (`multicast, vote`). Erhält er bzgl. der Quote oft genug die Zustimmung (`voteYes`), ist die Abstimmung positiv verlaufen und er meldet dies dem Koordinator. Als Kriterium für die Zustimmung gilt die Übereinstimmung mit dem jeweiligen internen $\#M_i$ Wert.

5

5

Aufgabenstellung

Beendigungsphase: Der Starter beendet sich nach der Initialisierungsphase (Start aller ggT-Prozesse) selbst. Die ggT-Prozesse werden vom Koordinator über die Beendigung des Systems informiert (`kill`). Stellt ein ggT-Prozess fest, dass er sich beenden soll (nur durch explizites kill-Kommando!), hat er dies unverzüglich zu tun, unabhängig von seinem aktuellen Zustand. Die einzige dann noch zulässige (und zwingend durchzuführende) Aktion ist, sich beim Namensdienst abzumelden (`unbind`). Hat der Koordinator alle ggT-Prozesse über die Beendigung informiert, meldet er sich beim Namensdienst ab und beendet sich. Der Namensdienst wird manuell beendet.

Starter: Der Starter (mit eindeutiger Nummer) erfragt beim Koordinator per Nachricht die steuernden Werte (`getsteeringval`). Der Starter liest aus der Datei `ggt.cfg` die weiteren Werte aus: die Erlang-Node des Namensdienstes (`nameservicenode`), den Namen des Koordinators (`koordinatorname`), die Nummer der Praktikumsgruppe (`praktikumsgruppe`) und die Nummer des Teams (`teamnummer`). Der Starter startet die ggT-Prozesse mit den zugehörigen Werten: die zufällig aus dem gegebenen Intervall gewählte Arbeitszeit, die Terminierungszeit, seine Startnummer dieses ggT-Prozesses, seine eindeutige Starternummer, die Praktikumsgruppennummer, die Teamnummer sowie die benötigten Kontaktdaten für den Namensdienst und den Koordinator. Beim starten des Starters wird ihm seine Starternummer mitgegeben.

Der Starter ist in Erlang/OTP zu implementieren. Ein ausführliches log, das die Aktivitäten detailliert darstellt, ist in einer separaten Datei zu erstellen. Als Dateien benötigt er lediglich seine eigene Datei, z.B. `ggtS.erl`, die der ggT-Prozesse, z.B. `ggt_process.erl`, und die Datei `ggt.cfg`.

6

6

Funktionalität

Koordinator: Die benötigten steuernden Werte liest der Koordinator aus der Datei `koordinator.cfg` aus. Diese umfassen die Erlang-Node des Namensdienstes (`nameservicenode`), die Anzahl der von einem Starter zu startenden ggT-Prozesse (`ggtprozessanzahl`), die Verzögerungszeit der ggT-Prozesse als simulierter Arbeitsaufwand für die Berechnung als Intervall (`arbeitszeit`) sowie die Terminierungszeit `?Xterm` (`termzeit`). Ebenso enthalten ist der Name des Koordinators (`koordinatorname`) sowie ein Flag, inwieweit der Koordinator bei Terminierungsmeldungen korrigierend eingreifen soll (`korrigieren`) und die für die Abstimmung benötigte Quote (`quote`) als prozentualer Wert. Der Koordinator registriert sich lokal bei der Erlang-Node und beim Namensdienst des Systems (`nameservice`). Nach dem Start des Koordinators und seiner Registrierung können Starter und ggT-Prozesse Nachrichten an ihn senden.

Ist der Koordinator im Zustand "initial", gibt er den Startern auf Anfrage die benötigten Informationen. Wird der Koordinator manuell in den Zustand "bereit" versetzt (`step`), gibt er keinem Starter mehr Auskunft und registriert keine ggT-Prozesse mehr. Er baut nun den Ring auf, indem er per Zufall die ggT-Prozesse in einem Ring anordnet. Erst danach geht er in den Zustand "bereit". Ist der Koordinator im Zustand "bereit" kann per manuellem Befehl (`{calc, WggT}`) eine ggT-Berechnung gestartet werden. Der Koordinator informiert dann alle ggT-Prozesse über deren jeweiligen Startwert `#Mi`. Zur Bestimmung dieser Werte verwendet er die Funktion `vsutil:bestimme_mis/2` zusammen mit dem gewünschten ggT (`WggT`). Dieser dient der Kontrolle der Berechnung.

7

7

Funktionalität

Er wählt dann per Zufall 20% aller ggT-Prozesse (mindestens jedoch zwei) aus, die mit der Berechnung beginnen sollen. Dazu sendet er diesen die Auswahlinformation zu (`calc, start`). Der jeweils ausgewählte ggT-Prozess erfragt dann den initialen Wert (`getinit`).

Der Koordinator wird von den ggT-Prozessen über deren Terminierung informiert. Sollte die dort gesendete Zahl größer als die bisher bekannte kleinste gemeldete Zahl sein, notiert der Koordinator diese Fehlmeldung in seiner log. Ist ein spezielles Flag gesetzt, sendet er dem ggT-Prozess diese kleinste Zahl per `sendy`. Achtung: beim Start einer neuen Berechnung muss dieser Wert neu initialisiert werden. Per manueller Eingabe (`kill`) kann der Koordinator in den Zustand "beenden" versetzt werden.

Ist der Koordinator im Zustand "beenden" informiert er die ggT-Prozesse über die Beendigung (`kill`) und meldet sich lokal bei seiner Erlang-Node (`unregister/1`) wie auch beim Namensdienst (`unbind`) ab.

Der Koordinator ist in Erlang/OTP zu implementieren und muss auf jedem Rechner im Labor startbar sein. Ein ausführliches log, das die Kommunikation mit den anderen Einheiten sowie die eigenen Aktivitäten detailliert darstellt, ist in einer separaten Datei zu erstellen. Als Dateien benötigt er lediglich seine eigene Datei, z.B. `koordinator.erl`, und die Datei `koordinator.cfg`.

8

8

Funktionalität

ggT-Prozess: Ein ggT-Prozess hat den Namen `?????`, wobei `?????` eine Zahl ist, die sich wie folgt zusammensetzt:

`<PraktikumsgruppenID><TeamID><Nummer des ggT-Prozess><Nummer des Starters>`,

z.B. ist in Praktikumsgruppe 4 von Team 03 ein zweiter ggT-Prozess von ihrem ersten Starter gestartet worden, so erhält dieser ggT-Prozess den Namen 4321. In der Kommunikation mit externen Prozessen sowie zur Registrierung beim Namensdienst wird dieser Name als atom verwendet. Lediglich die `From` in den Schnittstellenbeschreibungen dürfen PID's sein!

Der ggT-Prozess registriert sich lokal auf der Erlang-Node (`register`) und beim Namensdienst (`rebind`) mit seinem Namen. Er meldet sich dann beim Koordinator mit seinem Namen an (`hello`). Der ggT-Prozess erwartet dann vom Koordinator die Informationen über seine Nachbarn (`setneighbors`). Für eine neue ggT-Berechnung erwartet der ggT-Prozess vom Koordinator seine neue Zahl `#Mi` (`setpm`). Der ggT-Prozess kann zu jeder Zeit zu einer neuen Berechnung aufgefordert werden! Wenn der ggT-Prozess als Straterprozess ausgewählt wurde (`{calc, start}`-Nachricht), dann erfragt er beim Koordinator den ersten, initialen Wert zur Berechnung (`{getinit, self() }`). Dieser wird ihm dann mittels `sendy` mitgeteilt.

Wenn der ggT-Prozess innerhalb einer Berechnung eine Zahl erhält (`sendy`), führt er den ggT-Algorithmus aus. Ändert sich seine Zahl, informiert er per Nachrichten neben zwei zufälligen ggT-Prozessen im Ring (über den Namensdienst) zusätzlich den Koordinator, indem er diesem seinen Namen, seine neue Zahl und die aktuelle Systemzeit überträgt (`briefmi`). Ändert sich seine Zahl nicht wird keine Meldung vorgenommen.

9

9

Funktionalität

Für eine ggT-Berechnung braucht ein Prozess eine gewisse Zeit (die Arbeitszeit), die ihm vom Starter bei der Initialisierung mitgegeben wurde. Dies simuliert eine größere, Zeit intensivere Aufgabe. Der ggT-Prozess soll in dieser Zeit einfach nichts tun (`timer:sleep/1`); er muss auch keine anderen Nachrichten in dieser Zeit verarbeiten. Achtung: die Arbeitszeit wird in der `*.cfg` Datei in Sekunden angegeben. `timer:sleep/1` hat als Einheit Millisekunden.

Der ggT-Prozess beobachtet die Zeit seit dem letzten Empfang einer Zahl (`sendy` oder `setpm`). Hat diese `?Xterm` Sekunden überschritten (Terminierungszeit), startet er eine Terminierungsanfrage über den Namensdienst (`multicast, vote`). Es wird von ihm zu einer Zeit nur genau eine Terminierungsanfrage gestartet. Eine weitere kann frühestens dann gestartet werden, wenn zwischenzeitlich eine Zahl (`sendy, setpm`) an ihn gesendet wurde oder er mindestens zweimal `?Xterm` Sekunden lang auf die Zustimmung gewartet hat! Letzteres garantiert, dass er auf jeden Fall eine Terminierung melden wird. Bei dem Empfang einer neuen Zahl wird die aktuelle Terminierungsabfrage unmittelbar beendet. Ist die Terminierungsanfrage erfolgreich durchgeführt (`voteYes` ist zweimal eingegangen), sendet er dem Koordinator eine Mitteilung (`briefterm`) über die Terminierung der aktuellen Berechnung, die seinen Namen, den errechneten ggT (sein aktuelles `#Mi`) und seine aktuelle Systemzeit beinhaltet. Zudem zählt er seine erfolgreich gemeldeten Terminierungsmeldungen und notiert dies in seinem log.

10

10

Funktionalität

Wenn ein ggT-Prozess eine Anfrage nach der Terminierung (`vote`) erhält: stimmen seine $\#M_i$ Zahl und die in der `vote`-Nachricht enthaltenen $\#M_j$ -Zahl überein, dann antwortet er dem Initiator mit `voteYes` (explizites Zustimmung). Sonst ignoriert er die Nachricht (implizites ablehnen). Wenn das Flag zum Korrigieren gesetzt ist, informiert der ggT-Prozess den anfragenden Prozess über sein $\#M_i$, wenn dies kleiner als das zur Abstimmung gegebene M_j ist per `sendy`-Nachricht.

Der ggT-Prozess ist in Erlang/OTP zu implementieren und muss auf jedem Rechner im Labor startbar sein! Ein ausführliches log, das die Aktivitäten detailliert darstellt, ist in einer separaten Datei (pro ggT-Prozess!) zu erstellen. Als Dateien benötigt er lediglich seine eigene Datei, z.B. `ggt_process.erl`.

GUI:

Koordinator-GUI: Eine einfache Textausgabe auf dem Bildschirm und in einer separaten log-Datei. Alle ausgegebenen Informationen werden in der Datei `Koordinator@<Rechner>.log` protokolliert.

Starter-GUI: Eine einfache Textausgabe auf dem Bildschirm und in einer separaten log-Datei. Alle ausgegebenen Informationen werden in der Datei `ggtSTARTER_<NR>@<Rechner>.log` protokolliert, wobei `<NR>` die Nummer des zugehörigen Starters ist.

ggT-Prozess-GUI: Eine einfache Textausgabe auf dem Bildschirm und in einer separaten log-Datei. Alle ausgegebenen Informationen werden in der Datei `GGTP_<Name>@<Rechner>.log` protokolliert, wobei `<Name>` der Name des zugehörigen ggT-Prozesses ist.

11

11

Hinweise

Die Vorgaben sind so konzipiert, dass bestimmte, lehrreiche Effekte während einer Berechnung entstehen können. Bevor Sie hier versuchen, diese „Fehler“ zu vermeiden, fragen Sie bitte kurz nach, ob es sich um einen „Fehler“ handelt. Sie würden sonst gegen die Vorgabe implementieren!

Das System ist möglichst informativ zu gestalten, damit Sie diese lehrreichen Effekte auch entdecken können.

Die Schnittstelle zwischen dem Starter und den ggT-Prozessen ist nicht vorgegeben. Hier arbeiten nur Ihre eigenen Programme zusammen!

Veränderungen am Kernalgorithmus (Berechnung des ggT) sind nicht zulässig!

Die Konfigurationsdateien `*.cfg` sind nicht um weitere Attribute zu erweitern. Lediglich die Werte dürfen und sollten geändert werden.

In den Dateien `util.erl` und `vsutil.erl` gibt es für den Zeitstempel und loggen nützliche Funktionen.

Als Beispiel und zum Testen können Sie bitte das mit ausgelieferte System verwenden. Dies entbindet Sie jedoch nicht von eigenen Tests. Bei Fehlermeldungen aus dem gegebenen System heraus melden Sie sich bitte unmittelbar mit dem gesamten eigenen Code und den `*.cfg` Dateien, die eingesetzt wurden, dem Entwurf und der Fehlermeldung bei mir per E-Mail.

In den Dateien `util.erl` und `vsutil.erl` gibt es für den Zeitstempel und loggen nützliche Funktionen. Halten Sie das System einfach und strukturiert! Wenn die Kernfunktionalität erstellt wurde, kann das System immer noch beliebig erweitert/verbessert werden.

12

12

Beispiel: Verteilter Algorithmus

- ♦ **Satz von Euklid:** Der grösste gemeinsame Teiler (ggT) zweier positiver ganzer Zahlen x, y (mit $x \geq y > 0$) ist gleich dem ggT von y und dem Rest, der bei ganzzahliger Division von x durch y entsteht.
- ♦ **Eigenschaften:**
 - Man setzt nun noch $\text{ggT}(x, 0) := x$ für alle x
 - Rekursive Realisierung: $\text{ggT}(x, y) := \text{ggT}(y, \text{mod}(x, y))$
 - Offenbar ist $\text{ggT}(x, x) = x$ für alle x
- ♦ **Erweiterung:** $\text{mod}^*(x, y) := \text{mod}(x-1, y)+1$
- ♦ **Verteilter Algorithmus:**
 - Jeder Prozeß P_i hat seine eigene Variable M_i .
 - ggT aller am Anfang bestehender M_i wird berechnet:

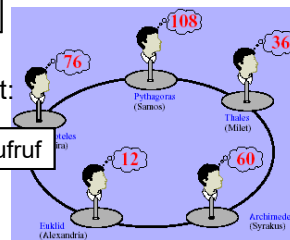

```
{Eine Nachricht <y> ist eingetroffen}
if y < M_i
then M_i := mod(M_i, y)+1;
send <M_i> to all neighbours;
fi
```

Noch nicht geklärt:

- Wie wird der Algorithmus gestartet ?
- Wie erkennt man die Terminierung ?
- Wo steht das Ergebnis ?

Modifikation für die Verteilung

Entspricht rekursivem Aufruf



13

13

Beispiel: Verteilter Algorithmus

- ♦ **Iterative Implementierung (für 2 Zahlen):**

```
While b <> 0 do
  Ersetze (a,b) durch (b, (a mod b))
return a
```

(76,12) → (12,4)
- ♦ **Rekursive Implementierung (für 2 Zahlen):**

```
Procedure GGT (a,b)
  if b == 0 return a
  else return GGT(b, (a mod b))
```

GGT(76,12) → GGT(12,4)

14

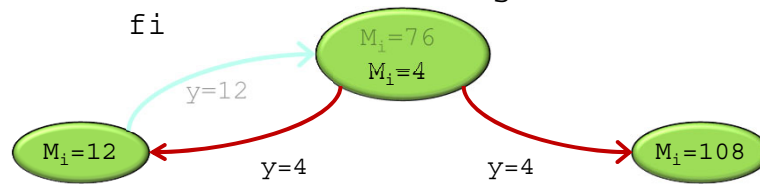
14

Beispiel: Verteilter Algorithmus

- Verteilte Implementierung (für n Zahlen):


```

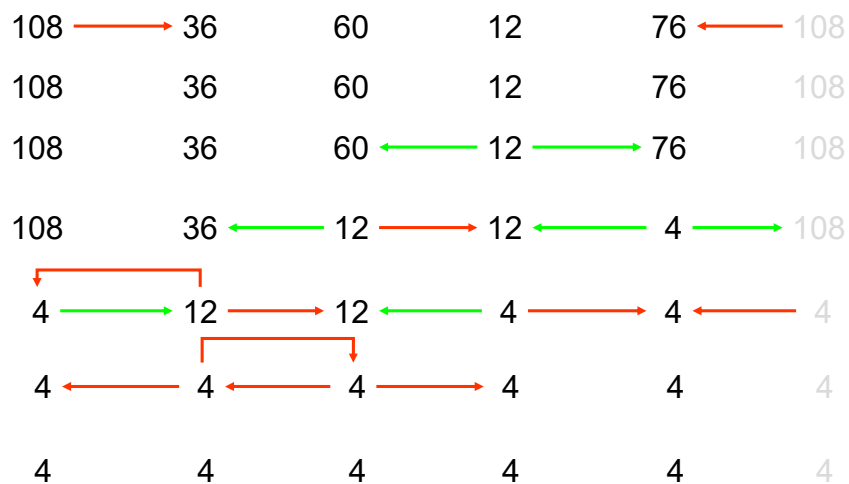
      {Eine Nachricht <y> ist eingetroffen}
      if y < Mi
        then Mi := mod(Mi-1, y)+1;
        send <Mi> to all neighbours;
      else do something else
      fi
      
```



15

15

Beispiel 1: Verteilter Algorithmus



16

16

Hinweis

41	21	11	41
15	12	6	15
6	12	6	6
6	6	6	6

Diagram illustrating a distribution process. A yellow header row contains values 41, 21, 11, and 41. Below, a 3x4 grid of numbers is shown. A green arrow points from the 6 in the second row, first column to the 12 in the second row, second column. A red arrow points from the 6 in the second row, fourth column to the 6 in the second row, third column. A red arrow points from the 15 in the first row, fourth column to the 6 in the second row, fourth column, with a green '9' next to it.

17

17

Beispiel 2: Verteilter Algorithmus

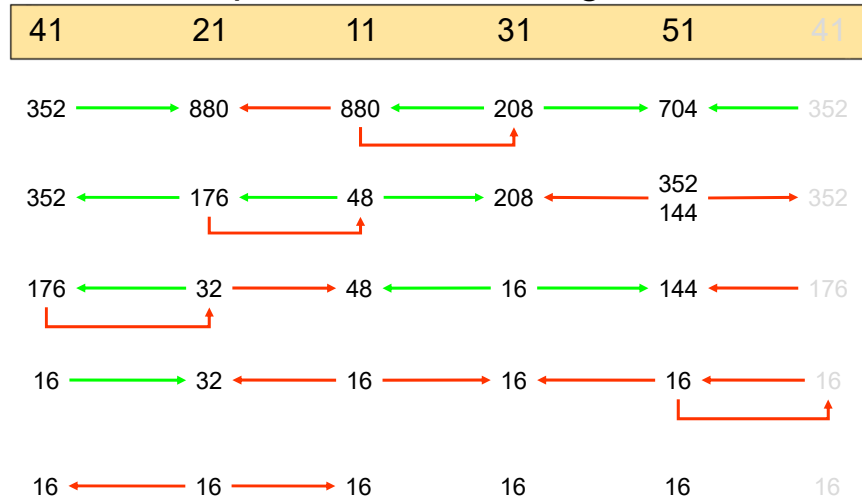
41	21	11	31	51	41
157.872	43.414.800	37.626.160	192.400	9.680	157872
					596336400
157.872	43.414.800	37.626.160	192.400	9.680	157872
157.872	43.414.800	37.626.160	192.400	9.680	157872
2.992	43.414.800	37.626.160	192.400	9.680	2992
2.992	880	37.626.160	192.400	704	2992
352	880	880	208	704	352

Diagram illustrating a distributed algorithm. A yellow header row contains values 41, 21, 11, 31, 51, and 41. Below, a 7x6 grid of numbers is shown. Green arrows indicate data flow: from 9.680 to 157872, from 157872 to 157872, from 157872 to 157872, from 2.992 to 43.414.800, from 43.414.800 to 37.626.160, from 37.626.160 to 192.400, from 192.400 to 9.680, from 9.680 to 2992, from 2.992 to 880, from 880 to 37.626.160, from 37.626.160 to 192.400, from 192.400 to 704, from 704 to 2992, from 352 to 880, from 880 to 880, from 880 to 208, from 208 to 704, from 704 to 352. A red arrow points from 9.680 to 596336400. A red arrow points from 2992 to 2992. A red arrow points from 880 to 208.

18

18

Beispiel 2: Verteilter Algorithmus



19

19

Verteilter Algorithmus

Ausschnitt aus der log-Datei

Koordinator-ko@Brummpa-KLC Startzeit: 17.06 10:13:52,969< mit PID <0.37.0>
 [...] Alle ggT-Prozesse gebunden.
 [...] Ring wird/wurde erstellt, Koordinator geht in den Zustand 'Bereit für Berechnung'.
 [...] Beginne eine neue ggT-Berechnung mit Ziel 16.
 ggT-Prozess 48851 (ggT@Brummpa) initiales Mi 9680 gesendet.
 [...] ggT-Prozess 48851 (ggT@Brummpa) startendes y 596336400 gesendet.
 [...] 48851 meldet Terminierung mit ggT 9680 um 17.06 10:15:52,000< (17.06 10:15:52,000<).
 [...] **Per Hand** 48841 die Zahl 9680 **gesendet** { , 48841, ggT@Brummpa } ! { sendy, 9680 }
 48841 meldet neues Mi 2992 um 17.06 10:18:36,499< (17.06 10:18:36,499<).
 48821 meldet neues Mi 880 um 17.06 10:18:39,100< (17.06 10:18:39,100<).
 48851 meldet neues Mi 704 um 17.06 10:18:39,100< (17.06 10:18:39,100<).
 48841 meldet neues Mi 352 um 17.06 10:18:42,700< (17.06 10:18:42,700<).
 48831 meldet neues Mi 208 um 17.06 10:18:42,700< (17.06 10:18:42,700<).
 48811 meldet neues Mi 880 um 17.06 10:18:42,700< (17.06 10:18:42,700<).
 48811 meldet neues Mi 48 um 17.06 10:18:45,200< (17.06 10:18:45,200<).
 48821 meldet neues Mi 176 um 17.06 10:18:45,200< (17.06 10:18:45,200<).
 ...

20

20

Verteilter Algorithmus

Ausschnitt aus der log-Datei

```
...
48851 meldet neues Mi 352 um 17.06 10:18:45,200< (17.06 10:18:45,200<).
48831 meldet neues Mi 16 um 17.06 10:18:48,800< (17.06 10:18:48,800<).
48821 meldet neues Mi 32 um 17.06 10:18:48,800< (17.06 10:18:48,800<).
48851 meldet neues Mi 144 um 17.06 10:18:48,801< (17.06 10:18:48,800<).
48841 meldet neues Mi 176 um 17.06 10:18:48,802< (17.06 10:18:48,800<).
48841 meldet neues Mi 16 um 17.06 10:18:51,300< (17.06 10:18:51,299<).
48811 meldet neues Mi 16 um 17.06 10:18:51,300< (17.06 10:18:51,300<).
48851 meldet neues Mi 16 um 17.06 10:18:51,300< (17.06 10:18:51,300<).
48821 meldet neues Mi 16 um 17.06 10:18:54,090< (17.06 10:18:54,090<).
48811 meldet Terminierung mit ggT 16 um 17.06 10:18:54,090< (17.06 10:18:54,090<).
48811 meldet Terminierung mit ggT 16 um 17.06 10:18:57,090< (17.06 10:18:57,090<).
48821 meldet Terminierung mit ggT 16 um 17.06 10:18:57,090< (17.06 10:18:57,090<).
48841 meldet Terminierung mit ggT 16 um 17.06 10:18:57,090< (17.06 10:18:57,090<).
[...]
```

21

21

Schnittstellen

Für den vorgegebenen **Namensdienst** (nameservice.beam) nameservice:

Nachdem ein ping auf den Knoten des Namensdienstes gemacht wurde (net_adm:ping(NameserviceNode)), erhält man die Adresse durch: Nameservice = global:whereis_name(nameservice), wobei nameservice der global registrierte Name des Namensdienstes ist. Beachten Sie bitte: dieses ping ist unbedingt auszuführen, obwohl in lokalen Netzen Erlang dies implizit durchführt. Im Labor funktioniert dies jedoch nicht!

Binden eines Dienstes (einmaliges binden):

```
register(meindienst,From),
Nameservice ! {From,{bind,meindienst,node()}},
receive ok -> io:format("..bind.done.\n");
        in_use -> io:format("..schon gebunden.\n")
end.
```

Rebind eines Dienstes (erstmaliges oder wiederholtes binden):

```
register(meindienst,From),
Nameservice ! {From,{rebind,meindienst,node()}},
receive ok_overwrite|ok_new -> io:format("..rebind.done.\n") end.
```

Reset vom Namensdienst:

```
Nameservice ! {From,reset},
receive ok -> do_something_else, end.
```

22

22

Schnittstellen

Lookup für einen Dienst:

```
Nameservice ! {From,{lookup,meindienst}},
receive not_found -> io:format("..meindienst..not_found.\n");
               {pin,{Name,Node}} -> io:format("...ok: {~p,~p}.\n",[Name,Node])
end.
```

Unbind eines Dienstes:

```
Nameservice ! {From,{unbind,meindienst}},
receive ok -> io:format("..unbind..done.\n") end,
unregister(meindienst).
```

Terminierungsabstimmung an alle registrierten Einheiten für die Wahl:

```
Nameservice ! {From,{multicast,vote,meinname,meinmi}},
receive {From,{vote,meinname,meinmi}} ->
        ...From ! {voteYes,Clientname}...,
end.
```

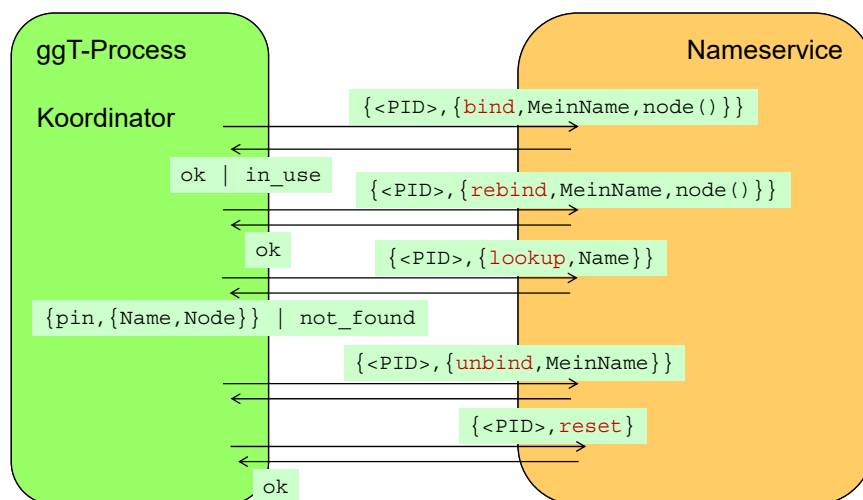
Quota setzen an alle registrierten Einheiten:

```
Nameservice ! {From,{multicast,quota,quota}},
receive {From,{setQuota,Quota}} -> ..., end
```

23

23

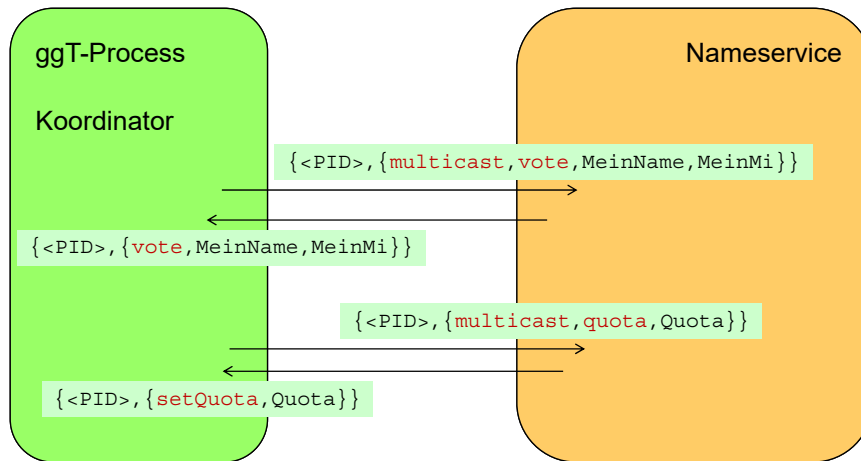
Verteilter Algorithmus: Namensdienst



24

24

Verteilter Algorithmus: Namensdienst



25

25

Schnittstellen

Starter Prozess:

- {steeringval, {AZMin, AZMax}, TermZeit, GGTProzessnummer}: die steuernden Werte für die ggT-Prozesse werden im Starter Prozess gesetzt; {AZMin, AZMax} ist die simulierte Verzögerungszeit zur Berechnung in Sekunden, TermZeit ist die Wartezeit in Sekunden, bis eine Wahl für eine Terminierung initiiert wird und GGTProzessnummer ist die Anzahl der zu startenden ggT-Prozesse.

ggT-Prozess:

- {setneighbors, LeftN, RightN}: die (lokal auf deren Node registrierten und im Namensdienst registrierten) Namen (keine PID!) des linken und rechten Nachbarn werden mitgeteilt.
- {setpm, MiNeu}: die von diesem Prozess zu bearbeitende Zahl für eine neue Berechnung wird mitgeteilt. Ggf. sind Informationen einer vorangegangenen Berechnung zu löschen.
- {sendy, Y}: der (rekursive) Aufruf der ggT-Berechnung.
- {calc, start}: der ggT-Prozess wurde als Start-Prozess ausgewählt. Mittels {getinit, From} holt sich der Prozess seinen ersten Wert zur Verrechnung mit seiner #Mi.

26

26

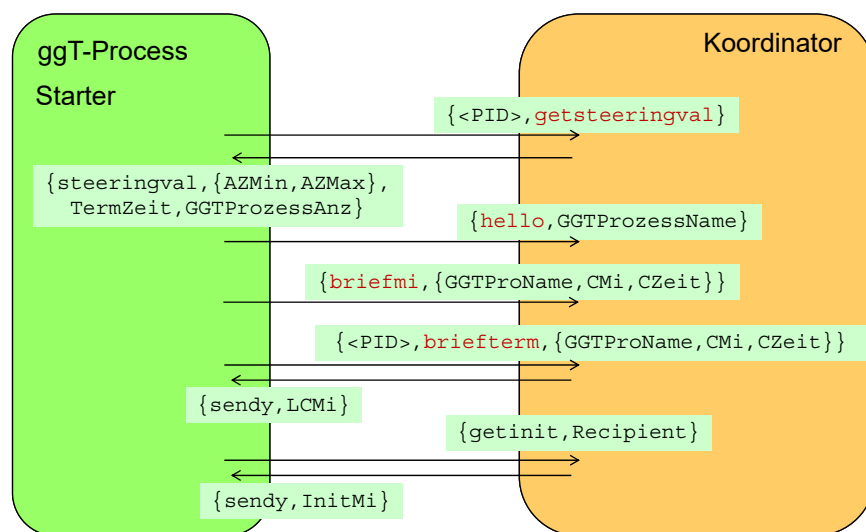
Schnittstellen

- {From,{vote,Initiator,MiIn}}: Wahlnachricht für die Terminierung der aktuellen Berechnung; Initiator ist der Initiator dieser Wahl (Name des ggT-Prozesses, keine PID!) und From (ist PID) ist sein Absender. MiIn ist das Mi des Initiators zur Klärung der Terminierungsfrage. Ist das Flag für die Korrektur gesetzt, sendet der ggT-Prozess per sendy an From sein #Mi, wenn #Mi < MiIn gilt.
- {voteYes,Name}: erhaltenes Abstimmungsergebnis, wobei Name der Name des Absenders ist (keine PID!).
- {From,toggle}: Der ggT-Prozess ändert sein Korrekturverhalten bei einer Abstimmungsanfrage.
- {From,tellmi}: Sendet das aktuelle Mi an From (ist PID): From ! {mi,Mi}. Wird vom Koordinator z.B. genutzt, um bei einem Berechnungsstillstand die Mi-Situation im Ring anzuzeigen.
- {From,pingGGT}: Sendet ein pongGGT an From (ist PID): From ! {pongGGT,GGTname}. Wird vom Koordinator z.B. genutzt, um auf manuelle Anforderung hin die Lebendigkeit des Rings zu prüfen.
- kill: der ggT-Prozess wird beendet.

27

27

Verteilter Algorithmus: Koordinator



28

28

Schnittstellen

Koordinator Prozess:

- {From:,getsteeringval} Die Anfrage nach den steuernden Werten durch den Starter Prozess (From ist seine PID).
- {hello,Clientname}: Ein ggT-Prozess meldet sich beim Koordinator mit Namen Clientname an (Name ist der lokal registrierte Name, keine PID!).
- {briefmi,{Clientname,CMi,CZeit}}: Ein ggT-Prozess mit Namen Clientname (keine PID!) informiert über sein neues Mi CMi um CZeit Uhr (CZeit wird mittels util:timeMilliSecond() erstellt!).
- {From,briefterm,{Clientname,CMi,CZeit}}: Ein ggT-Prozess mit Namen Clientname (keine PID!) und Absender From (ist PID) informiert über die Terminierung der Berechnung mit Ergebnis CMi um CZeit Uhr.
- {getinit,From}: Ein ggT-Prozess mit Absender From (ist PID) fragt einen Wert zum Start der Berechnung nach. Der Koordinator sendet ihm diesen Wert per sendy-Nachricht.
- {calc,WggT}: Der Koordinator startet eine neue ggT-Berechnung mit Wunsch-ggT WggT.

29

29

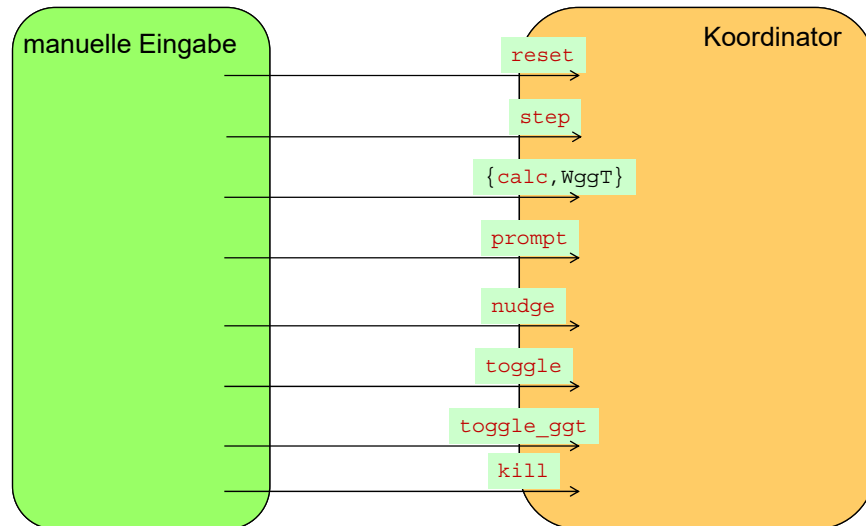
Schnittstellen

- reset: Der Koordinator sendet allen ggT-Prozessen das kill-Kommando und bringt sich selbst in den initialen Zustand, indem sich Starter wieder melden können.
- step: Der Koordinator beendet die Initialphase und bildet den Ring. Er wartet dann auf den Start einer ggT-Berechnung.
- prompt: Der Koordinator erfragt bei allen ggT-Prozessen per tellmi-Nachricht deren aktuelles Mi ab und zeigt dies im log an.
- nudge: Der Koordinator erfragt bei allen ggT-Prozessen per pingGGT-Nachricht deren Lebenszustand ab und zeigt dies im log an.
- toggle: Der Koordinator verändert den Flag zur Korrektur bei falschen Terminierungsmeldungen.
- toggle_ggt: Der Koordinator verändert den Flag zur Korrektur bei vote-Anfragen bei allen ggT-Prozessen mittels toggle-Nachricht.
- kill: Der Koordinator wird beendet und sendet allen ggT-Prozessen das kill-Kommando.

30

30

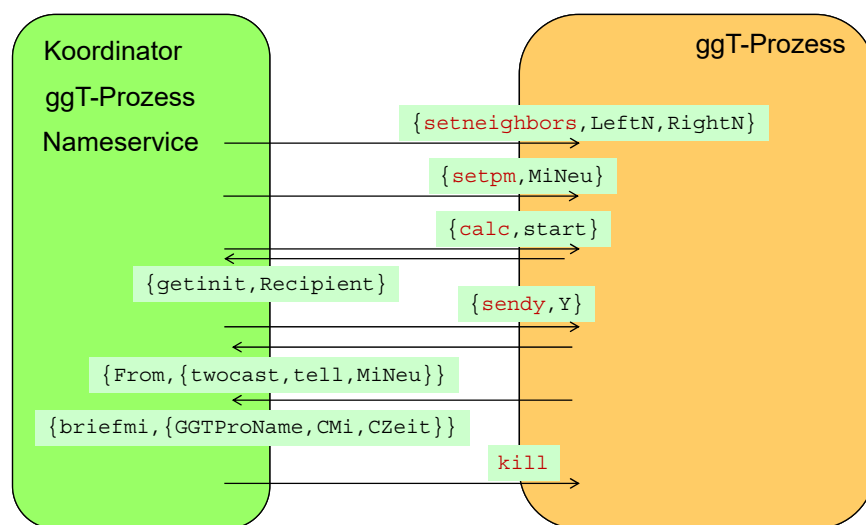
Verteilter Algorithmus: Koordinator



31

31

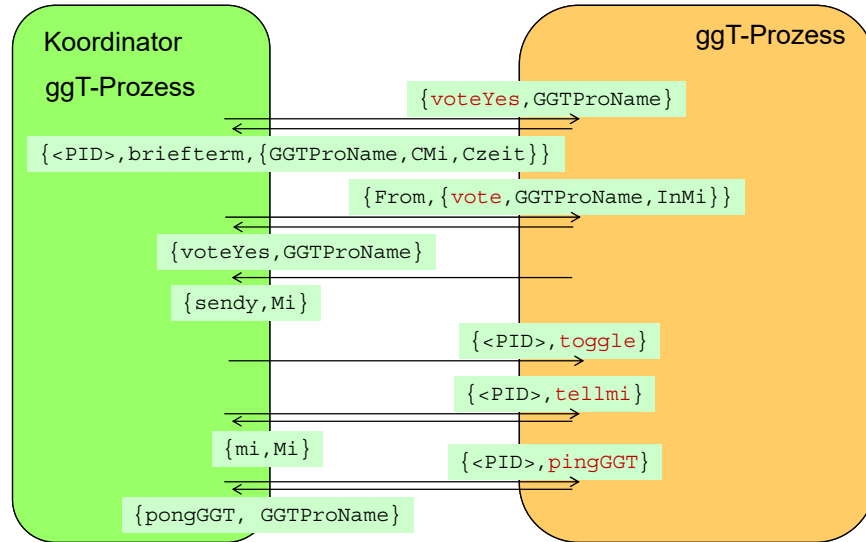
Verteilter Algorithmus: ggT-Prozess



32

32

Verteilter Algorithmus: ggT-Prozess



33

33

Namensdienst:
lab22

Anfang

```
(w)erl -s name auskunft (-setcookie zummsel)
1> nameservice:start().
%global:register_name(nameservice, NServerPid)
```

34

34

Anfang

Namensdienst:
lab22


```
NameS ! {self(), {rebind, KoName, KoNode}}
```

Koordinator
lab24

```
(w)erl -s name chief (-setcookie zummsel)
1>koordinator:start().
%net_adm:ping(NameServiceNode)
%register(KoName, KoPID)
```

35

35

Praxis

```
Eshell V13.1.2 (abort with ^G)
(chief@Qigong-KLC)1> koordinator:start().
Koordinator-chief@Qigong-KLC-KLC Startzeit: 12.01 11:33:53,735| mit PID <0.87.0>
koordinator.cfg geöffnet...
koordinator.cfg gelesen...
Nameservice nameservice auf 'auskunft@Qigong-KLC' mit PID <9501.89.0> gebunden...
ok
lokal mit euklid registriert...
beim Namensdienst mit Node 'chief@Qigong-KLC' neu registriert.
(chief@Qigong-KLC)2>
```

```
Eshell V13.1.2 (abort with ^G)
(auskunft@Qigong-KLC)1> nameservice:start().
Nameservice Startzeit: 12.01 11:33:34,808| mit PID <0.89.0> registriert mit Namen nameservice.
ok
rebind um 12.01 11:33:54,293| von <9647.87.0> (euklid):{euklid,chief@Qigong-KLC}.
(auskunft@Qigong-KLC)2> ns_send:msg(list,node()).
..listall: [{euklid,'chief@Qigong-KLC'}] ..done.
listall um 12.01 11:34:44,430| von <0.87.0>.
ok
(auskunft@Qigong-KLC)3>
```

36

36

Anfang

Namensdienst:
lab22Koordinator
lab24

Names ! {self(), {rebind, Clientname, self()}}

Koordinator ! {hello, Clientname}

Starter 1, Gruppe 08

Starter 2, Gruppe 08

```
(w)erl - (s)name ggT (-setcookie zummsel)
1>ggTS:go(Anzahl,Start).
%net_admin:ping(NameServiceNode)
%register(Clientname,self())
```

488311

37

37

Praxis

```
Erlang/OTP 25 [erts-13.1.2] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit:ns]

Eshell V13.1.2 (abort with ^G)
(ggTA@Qigong-KLC)1> ggTS:go(2,1).
Starter_1-ggTA@Qigong-KLC-KLC Startzeit: 12.01 11:37:25,329| mit PID <0.87.0>
ggT.cfg geöffnet...
ggT.cfg gelesen...
Nameservice nameservice auf 'auskunft@Qigong-KLC' gebunden...
Koordinator euklid (euklid) auf 'chief@Qigong-KLC' gebunden.
getsteeringval: 7 bis 13 Arbeitszeit ggT; 3 Wartezeit Terminierung ggT; 9-te GGT Prozess
Starter_2-ggTA@Qigong-KLC-KLC Startzeit: 12.01 11:37:25,812| mit PID <0.87.0>
48841 Startzeit: 12.01 11:37:25,818| mit PID <0.110.0> auf ggTA@Qigong-KLC und Arbeitsze
it 13000ms.
beim Namensdienst und auf Node lokal neu registriert.
beim Koordinator gemeldet.
48821 Startzeit: 12.01 11:37:25,818| mit PID <0.112.0> auf ggTA@Qigong-KLC und Arbeitsze
it 10000ms.
beim Namensdienst und auf Node lokal neu registriert.
beim Koordinator gemeldet.
48891 Startzeit: 12.01 11:37:25,818| mit PID <0.105.0> auf ggTA@Qigong-KLC und Arbeitsze
it 10000ms.
```

38

38

Praxis

```

beim Namensdienst und auf Node lokal neu registriert.
beim Koordinator gemeldet.
48864 Startzeit: 12.01 11:37:34,237| mit PID <0.147.0> auf ggtB@Qigong-KLC und Arbeitszeit
13000ms.
beim Namensdienst und auf Node lokal neu registriert.
beim Koordinator gemeldet.
48824 Startzeit: 12.01 11:37:34,237| mit PID <0.151.0> auf ggtB@Qigong-KLC und Arbeitszeit
13000ms.
beim Namensdienst und auf Node lokal neu registriert.
48814 Startzeit: 12.01 11:37:34,236| mit PID <0.152.0> auf ggtB@Qigong-KLC und Arbeitszeit
9000ms.
beim Namensdienst und auf Node lokal neu registriert.
beim Koordinator gemeldet.
beim Koordinator gemeldet.
48854 Startzeit: 12.01 11:37:34,237| mit PID <0.148.0> auf ggtB@Qigong-KLC und Arbeitszeit
13000ms.
beim Namensdienst und auf Node lokal neu registriert.
beim Koordinator gemeldet.
48844 Startzeit: 12.01 11:37:34,237| mit PID <0.149.0> auf ggtB@Qigong-KLC und Arbeitszeit
11000ms.
beim Namensdienst und auf Node lokal neu registriert.
beim Koordinator gemeldet.
(ggtB@Qigong-KLC)2>

```

39

39

Praxis

```

getsteeringval: <9725.87.0> (9).
hello: 48841 (9).
hello: 48821 (9).
hello: 48891 (9).
hello: 48881 (9).
hello: 48861 (9).
hello: 48851 (9).
hello: 48831 (9).
hello: 48811 (9).
hello: 48871 (9).
getsteeringval: <9725.87.0> (18).
hello: 48842 (18).
hello: 48822 (18).
hello: 48892 (18).
hello: 48882 (18).
hello: 48872 (18).
hello: 48862 (18).
hello: 48852 (18).
hello: 48832 (18).
hello: 48812 (18).
getsteeringval: <9744.87.0> (27).
hello: 48883 (27).
hello: 48873 (27).
hello: 48853 (27).
hello: 48893 (27).
hello: 48863 (27).
(auskunft@Qigong-KLC)3> ns_send:msg(list,node()).
..listall: [{'48864','ggtB@Qigong-KLC'},
{'48851','ggtA@Qigong-KLC'},
{'48884','ggtB@Qigong-KLC'},
{'48833','ggtB@Qigong-KLC'},
{'48843','ggtB@Qigong-KLC'},
{'48853','ggtB@Qigong-KLC'},
{'48874','ggtB@Qigong-KLC'},
{'48831','ggtA@Qigong-KLC'},
{'48841','ggtA@Qigong-KLC'},
{'48814','ggtB@Qigong-KLC'},
{'48832','ggtA@Qigong-KLC'},
{'48872','ggtA@Qigong-KLC'},
{'48842','ggtA@Qigong-KLC'},
{'48892','ggtA@Qigong-KLC'},
{'48892','ggtA@Qigong-KLC'}].
lookup um 12.01 11:37:33,787| von <9687.87.0>:{euklid,chief@Qigong-KLC}.
rebind um 12.01 11:37:33,801| von <9687.111.0> ('48883'):{'48883','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:33,802| von <9687.112.0> ('48873'):{'48873','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:33,803| von <9687.114.0> ('48853'):{'48853','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:33,805| von <9687.116.0> ('48833'):{'48833','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:33,805| von <9687.110.0> ('48893'):{'48893','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:33,805| von <9687.113.0> ('48863'):{'48863','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:33,805| von <9687.115.0> ('48843'):{'48843','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:33,805| von <9687.117.0> ('48823'):{'48823','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:33,806| von <9687.118.0> ('48813'):{'48813','ggtB@Qigong-KLC'}.
lookup um 12.01 11:37:34,223| von <9687.87.0>:{euklid,chief@Qigong-KLC}.
rebind um 12.01 11:37:34,231| von <9687.150.0> ('48834'):{'48834','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:34,231| von <9687.144.0> ('48894'):{'48894','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:34,231| von <9687.145.0> ('48884'):{'48884','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:34,231| von <9687.146.0> ('48874'):{'48874','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:34,232| von <9687.152.0> ('48814'):{'48814','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:34,232| von <9687.148.0> ('48854'):{'48854','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:34,232| von <9687.147.0> ('48864'):{'48864','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:34,232| von <9687.149.0> ('48844'):{'48844','ggtB@Qigong-KLC'}.
rebind um 12.01 11:37:34,232| von <9687.151.0> ('48824'):{'48824','ggtB@Qigong-KLC'}.
(auskunft@Qigong-KLC)3>

```

40

40

Anfang

Namensdienst:
lab22

Koordinator
lab24

488411
lab31

488211
lab32

488311
lab23

41

41

Praxis

```
(chief@Qigong-KLC)2> ko_send:msg(step).
..chef..ok: {euklid,'chief@Qigong-KLC'}.
..step..done.
Anmeldefrist fuer ggT-Prozesse abgelaufen. Vermisst werden 0 ggT-Prozesse.
Anzahl der Startprozesse:7.
done
ggT-Prozess 48844 (48844) auf ggTB@Qigong-KLC gebunden.
ggT-Prozess 48854 (48854) auf ggTB@Qigong-KLC gebunden.
ggT-Prozess 48824 (48824) auf ggTB@Qigong-KLC gebunden.
ggT-Prozess 48841 (48841) auf ggTA@Qigong-KLC gebunden.
Alle ggT-Prozesse gebunden.
Alle ggT-Prozesse ueber Nachbarn informiert.
Ring wird/wurde erstellt, Koordinator geht in den Zustand 'Bereit fuer Berechnung'.
ggT-Prozess 48873 (ggTB@Qigong-KLC) ueber linken (48861) und rechten (48811) Nachbarn informiert.
ggT-Prozess 48811 (ggTA@Qigong-KLC) ueber linken (48873) und rechten (48874) Nachbarn informiert.
ggT-Prozess 48874 (ggTB@Qigong-KLC) ueber linken (48811) und rechten (48843) Nachbarn informiert.
ggT-Prozess 48843 (ggTB@Qigong-KLC) ueber linken (48874) und rechten (48892) Nachbarn informiert.
Linker Nachbar 48871 (48871) gebunden.
Rechter Nachbar 48844 (48844) gebunden.
Linker Nachbar 48884 (48884) gebunden.
Rechter Nachbar 48894 (48894) gebunden.
Linker Nachbar 48881 (48881) gebunden.
Rechter Nachbar 48873 (48873) gebunden.
Linker Nachbar 48842 (48842) gebunden.
Rechter Nachbar 48861 (48861) gebunden.
(ggTA@Qigong-KLC)2>
```

42

42

Praxis

```

ggT-Prozess 48801 (ggTA@Qigong-KLC) überlinken (48881) und rechnen
(chief@Qigong-KLC)3> ko_send:msg({calc,7}).
..chef..ok: {euklid,'chief@Qigong-KLC'}.
..calc 7..done.
Beginne eine neue ggT-Berechnung mit Ziel 7.
done
ggT-Prozess 48841 (ggTA@Qigong-KLC) initiales Mi 23195718 gesendet.
ggT-Prozess 48821 (ggTA@Qigong-KLC) initiales Mi 150150 gesendet.
ggT-Prozess 48891 (ggTA@Qigong-KLC) initiales Mi 444675 gesendet.

48894 meldet neues Mi 42 um "12.01 11:47:27,949|" (12.01 11:47:27,954|).
..m42 48883 meldet neues Mi 21 um "12.01 11:47:30,945|" (12.01 11:47:30,950|).
48811 meldet neues Mi 182 um "12.01 11:47:31,933|" (12.01 11:47:31,945|).
48871 meldet neues Mi 42 um "12.01 11:47:38,947|" (12.01 11:47:38,958|).
48812 meldet neues Mi 70 um "12.01 11:47:41,943|" (12.01 11:47:41,955|).
48882 meldet neues Mi 21 um "12.01 11:47:42,941|" (12.01 11:47:42,952|).
48854 meldet neues Mi 21 um "12.01 11:47:43,961|" (12.01 11:47:43,966|).
(chief@Qigong-KLC)4>

48821: stimme ab (48832): mit >NEIN< gestimmt und ignoriert.
sendy: 854 (12728716,true|7); berechnet als neues Mi 700. 12.01
48872: stimme ab (48841): mit >NEIN< gestimmt und ignoriert.
sendy: 700 (77175,false|7); keine Berechnung (false|7).
(ggTA@Qigong-KLC)2>

```

```

twocast tell (<9668.147.0>) mit 7 an {'48842'},
twocast tell (<9687.150.0>) mit 14 an {'48852'},
twocast tell (<9687.113.0>) mit 7 an {'48821'},
twocast tell (<9687.116.0>) mit 7 an {'48844'},
twocast tell (<9687.145.0>) mit 14 an {'48862'},
twocast tell (<9687.114.0>) mit 7 an {'48881'},
twocast tell (<9687.146.0>) mit 7 an {'48894'},
twocast tell (<9668.144.0>) mit 7 an {'48811'},
twocast tell (<9668.139.0>) mit 7 an {'48871'},
twocast tell (<9668.112.0>) mit 7 an {'48882'},
twocast tell (<9687.110.0>) mit 7 an {'48861'},
(auskunft@Qigong-KLC)4>

```

43

43

Durchzuführende Tests

Test 1: **Ihr Koordinator**, vorgegebene ggT Prozesse

Test 2: Vorgegebener Koordinator, **Ihre ggT Prozesse**

Test 3: **Ihr Koordinator**, vorgegebene und **Ihre ggT Prozesse**

Test 4: Vorgegebener Koordinator, vorgegebene und **Ihre ggT Prozesse**

In koordinator.cfg

{arbeitszeit, {7,13}}. Intervall variieren, Effekte beobachten
 {termzeit, 3}. „Geduld“ variieren, Effekte beobachten
 {korrigieren, 0}. Koordinator oder/und ggT-Prozesse korrigieren lassen
 und nicht korrigieren lassen, Effekte beobachten
 Achtung: es kann sein, dass je nach Einstellung nicht alle zum selben
 Ergebnis kommen!

```

GGTP_48801@Qigong-KLC.log
GGTP_48882@Qigong-KLC.log
GGTP_48883@Qigong-KLC.log
GGTP_48884@Qigong-KLC.log
GGTP_48891@Qigong-KLC.log
GGTP_48892@Qigong-KLC.log
GGTP_48893@Qigong-KLC.log
GGTP_48894@Qigong-KLC.log
ggTSTARTER_1@Qigong-KLC.log
ggTSTARTER_2@Qigong-KLC.log
ggTSTARTER_3@Qigong-KLC.log
ggTSTARTER_4@Qigong-KLC.log
Koordinator@Qigong-KLC.log
nameservice@Qigong-KLC.log

```

44

44

Abnahme

- ♦ Bis Donnerstagabend 20:00 Uhr, eine Woche vor dem Praktikumstermin ist bitte der Entwurf für die Aufgabe als *.pdf Dokument (Dokumentationskopf nicht vergessen!) mir per E-Mail (mit cc an den/die Teampartner_in) zuzusenden. **Beachten Sie die Vorgaben zu einem Entwurf!**
- ♦ Am Abend vor dem Praktikum ist bis 20:00 Uhr bitte die finale Abgabe als *.zip Dokument einzusenden: bitte finaler Stand (als *.zip mit cc an Teampartner_in) per E-Mail abgeben, bestehend aus
 - dem von Ihnen entwickeltem Code zur Lösung der Aufgabe (bitte am Anfang des Codes Ihren Namen explizit aufführen: `-author („Name“)`), der alle Vorgaben erfüllen muss. In den Sourcedateien ist auf den Entwurf zu verweisen, um die Umsetzung der Vorgaben zu dokumentieren.
 - den log-Dateien sowie der Bildschirmausgabe mit dem vorgegebenem Testsystem. Dieser Test entbindet Sie jedoch nicht von eigenen Tests. Bei Fehlermeldungen aus dem gegebenen System melden Sie sich bitte unmittelbar mit dem gesamten eigenen Code und den *.cfg Dateien, die eingesetzt wurden, dem Entwurf und der Fehlermeldung bei mir per E-Mail.Folgende Tests sind bitte durchzuführen:
 - Test 1: **Ihr Koordinator**, vorgegebene ggT Prozesse
 - Test 2: Vorgegebener Koordinator, **Ihre ggT Prozesse**
 - Test 3: **Ihr Koordinator**, vorgegebene und **Ihre ggT Prozesse**
 - Test 4: Vorgegebener Koordinator, vorgegebene und **Ihre ggT Prozesse**

45

45

Abnahme

- ♦ Analyse: Ergebnisse von Variationen der Daten in der `koordinator.cfg`.
- ♦ Die Abgabe wird als Abgabe gewertet, wenn der zugehörige Entwurf mindestens als ausreichend bewertet wurde und diese die beschriebenen Tests erfolgreich durchläuft. Dies sichert jedoch keine erfolgreiche Abnahme zu.
- ♦ Am Tag des Praktikums findet eine gemeinsame Vorführung und eine Besprechung mit den Teams statt. Die Besprechung muss erfolgreich absolviert werden, um weiter am Praktikum teilnehmen zu können. Bei der Besprechung handelt es sich nicht um die Abnahme!

46

46

Algorithmen und Datenstrukturen

Entwurf

47

47

Clean Code Developer

Sollte vielleicht **auf Entwurf verzichtet werden**, wenn letztlich in der Implementation die „Strukturwahrheit“ liegt? **Nein, sicher nicht**. Entwurf muss sein. **Ohne Planung gibt es keine Zielvorstellung**. Aber Entwurf und Implementation müssen dem „Don't Repeat Yourself“-Prinzip gerecht werden. Deshalb sollten Entwurf und Implementation sich so wenig überlappen wie möglich. [...] Wenn das der Fall ist, stellen sie keine Wiederholungen mehr dar, sondern beschreiben unterschiedliches. Das bedeutet: Entwurf/Architektur kümmert sich nicht um die Implementation und Implementation kümmert sich nicht um Architektur.

Und wo verläuft diese Trennlinie? Bei den so genannten Komponenten. Architekten kümmern sich nicht um den internen Aufbau von Komponenten. Für sie sind es Black Boxes, deren Klassenstruktur nicht architekturelevant ist. Umgekehrt ist für einen Komponentenimplementierer die Architektur irrelevant. Was er zu implementieren hat, ergibt sich aus den Komponentenkontrakten, die seine Komponente importiert und exportiert. Einen größeren Zusammenhang muss er nicht kennen.

Die Aufgabe der Architektur ist es mithin, Software in Komponenten zu zerlegen, deren Abhängigkeiten zu definieren und Leistungen in Kontrakten zu beschreiben. [...] Und die Aufgabe der Implementation ist es, die von der Architektur definierten Komponenten zu realisieren. Wie sie das tun, ist nicht architekturelevant. Ihre innere Struktur ist für die Architektur unsichtbar.

48

48

Entwurf: hier



- Der Entwurf enthält eine detaillierte Beschreibung der benötigten Daten in den jeweiligen Schritten der Lösungs-Algorithmen (**Lösungs-Datenstrukturen**). Es wird detailliert beschrieben, welche Daten in welcher Form in den jeweiligen Schritten benötigt werden. Die Beschreibung orientiert sich an den inhaltlichen/funktionalen Anforderungen. Die Datenstrukturen sind explizit vor den Algorithmen zu beschreiben.
- Der Entwurf enthält die detaillierte Beschreibung der inhaltlichen/funktionalen Aufgaben/Abläufe des Problems (**Lösungs-Algorithmen**) und ggf. der verwendeten Datenstrukturen. Jeder Schritt ist im Detail durchdacht, d.h. es gibt bzgl. der inhaltlichen/funktionalen Aufgabe keine offenen Fragen mehr! Darstellungsvorgabe: Diagramme gemäß Vorgabe auf nächster Folie plus ggf. Erklärungen/Erläuterungen.
- Der Entwurf enthält die Beschreibung aller wichtigen Schnittstellen sowie Vorgaben aus der Aufgabenstellung und beschreibt damit auch im Detail die Architektur des Systems (**Lösungs-Architektur**).
- Der Entwurf ist **vollständig in seiner Beschreibung**, d.h. für die technische Umsetzung (Implementierung) ist der Entwurf als einziges Dokument (auch für andere Teams) ausreichend
- Der Entwurf **beschreibt nicht die technische Umsetzung**, also die Art und Weise, wie die beschriebenen, Problem orientierten Algorithmen in eine Programmiersprache effizient und korrekt umgesetzt werden. Daher ist der Entwurf unabhängig von der gewählten Programmiersprache (außer den ggf. gegebenen technischen Vorgaben)

49

49

Entwurf: hier

- Für die lokalen Algorithmen: Vorgabe für Ablaufdiagramme DIN 66001 (Sinnbilder und ihre Anwendung): z.B. <https://akademie.123effizientdabei.de/wp-content/uploads/2023/02/DIN66001-1966.pdf> mit z.B. draw.io (<https://app.diagrams.net/>)
- Auch möglich (Für die verteilten Algorithmen geeignet): Sequenzdiagramme nach UML: z.B. GitMind (<https://gitmind.com/de/>), Lucidchart (<https://www.lucidchart.com/pages/>) oder StarUML (<https://staruml.io/>)
- In den Diagrammen sind Zeichen (z.B. Zahlen/Schlüsselwörter etc.) zu verwenden, um so aus dem Code heraus auf diese Stellen im Entwurf verweisen zu können.

50

50

Entwurf: Gliederung

1) Einleitung

Überblick über die Aufgabenstellung

2) Anforderungen

- i. Funktionale Anforderungen
- ii. Technische Anforderungen

3) Datenstrukturen

Beschreibung der in den Algorithmen verwendeten Datenstrukturen. Begründung, warum diese für die Aufgabenstellung geeignet sind und warum diese die Bearbeitung fördern.

4) Detaillierte Algorithmen

Detaillierte Beschreibung (in einem Ablauf-/Flussdiagramm) der Algorithmen. Passende Erläuterungen des Pseudocodes in den Diagrammen. Komplexitätsanalyse bzgl. Zeit zur Bewertung der Leistungsfähigkeit. Hinweis auf spezielle Fälle (z.B. Randfälle) und Fehlerbehandlung. Falls hilfreich, Beispiele bzgl. Eingabe und Ausgabe.

5) Testplan

- i. Teststrategien, Testfälle und Szenarien für die funktionalen Tests
- ii. Laufzeittests beschreiben: Ziel der Tests, Hardware, Software, Testdaten (Menge und Struktur), Testfälle, Testdurchführung, erwartete Ergebnisse